



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Laboratorio 2: APIs REST con Spring Boot, Componentización con Web Components y CSS Moderno

Metadatos

- **Tiempo Estimado:** 5 horas (trabajo extraclase autónomo)
- **Herramientas Requeridas:**
 - JDK 17 o superior (instalado localmente).
 - Maven 3.8+ o Maven Wrapper.
 - VS Code con extensiones: Java Extension Pack y Spring Boot Extension Pack.
 - Navegador web moderno con herramientas de desarrollo (DevTools).
- **Metas de Aprendizaje:**

- Diseñar y construir un API REST con controladores de Spring Boot (@RestController) que exponga datos y realice validaciones.
- Desarrollar componentes web nativos encapsulados con Shadow DOM (<template>, <slot>) y con manejo dinámico de atributos.
- Crear interfaces responsivas utilizando variables de CSS, selectores anidados (Nesting) y Container Queries (@container).

Introducción

En el desarrollo de software full-stack moderno, las aplicaciones se diseñan bajo una arquitectura desacoplada. El servidor (Backend) actúa únicamente como un proveedor de recursos y servicios de datos de tipo RESTful, mientras que la interfaz (Frontend) es independiente y consume los servicios de forma asíncrona.

En este laboratorio calificado, usted construirá una aplicación de **Gestor de Biblioteca Personal**. Esta aplicación permitirá registrar libros, verlos en una cuadrícula autoajutable y catalogarlos como leídos o no leídos.

Usted iniciará a partir de la estructura del proyecto suministrada y deberá implementar la funcionalidad requerida aplicando los conocimientos de arquitectura en capas con Spring Boot y componentes del navegador nativos.

Parte 1: Estructura del Proyecto

En el directorio del laboratorio se incluye el siguiente esqueleto de archivos, el cual debe ser utilizado como base:

```
Laboratorio_2/  
  ■■■ Laboratorio_2.md  
  ■■■ libros-seed.json  
  ■■■ backend/  
    ■ ■■■ pom.xml  
    ■ ■■■ src/main/java/cr/ac/ucr/paraiso/ie/carnet/laboratorio2/
```

```
■ ■ ■ ■ Laboratorio2Application.java
■ ■ ■ ■ frontend/
■ ■ ■ ■ index.html
■ ■ ■ ■ style.css
■ ■ ■ ■ components/
■ ■ ■ ■ TarjetaLibro.js
■ ■ ■ ■ PanelBiblioteca.js
```

[!IMPORTANT] Recuerde que para respetar el estándar de nomenclatura del curso, debe renombrar la carpeta del paquete carnet y modificar la declaración package del archivo Laboratorio2Application.java con su carné de estudiante real en minúsculas (ej. b98765).

Parte 2: Especificaciones del Backend (Spring Boot)

El backend de la aplicación debe estructurarse utilizando el patrón de arquitectura por capas. Deberá crear los siguientes componentes:

1. Modelo de Datos (domain/Libro.java)

Cree la clase de dominio Libro con los siguientes campos y encapsulación adecuada:

- `id` (int): Identificador numérico autoincremental en memoria.
- `titulo` (String): Título del libro (Obligatorio, no vacío).
- `autor` (String): Autor del libro (Obligatorio, no vacío).
- `categoria` (String): Categoría del libro (Novela, Técnico, etc.).
- `anioPublicacion` (int): Año de publicación. No debe ser mayor al 2026.
- `leido` (boolean): Estado indicativo si ya finalizó la lectura.
- `resena` (String): Comentario u opinión personal sobre el libro.

2. Controlador REST (controller/LibroController.java)

Cree un controlador REST anotado con `@RestController` que exponga la ruta base `/api/libros`. Este controlador debe cumplir con:

- **Simulación en memoria:** Gestione una colección `List<Libro>` para almacenar las instancias en tiempo de ejecución.
- **Carga de Datos Semilla:** En el constructor del controlador, cargue al menos 3 libros con los datos definidos en el archivo `libros-seed.json` para pre-poblar el catálogo.
- **Habilitar CORS:** Agregue `@CrossOrigin(origins = "*")` para permitir las peticiones asíncronas desde el puerto donde ejecute el frontend.
- **Endpoint GET (/api/libros):** Retorna la lista completa de libros con código HTTP `200 OK`.
- **Endpoint POST (/api/libros):** Recibe los datos en formato JSON (`@RequestBody`) y agrega un nuevo libro:
 - Debe validar que el título y el autor no sean nulos o cadenas vacías. De ser así, retorne código HTTP `400 Bad Request` y un mensaje de error legible.
 - Debe validar que el `anioPublicacion` no sea mayor a 2026. De ser así, retorne código HTTP `400 Bad Request` y un mensaje de error explícito.
 - Asigne el `id` dinámicamente y agregue el elemento a la lista. Retorne el libro creado con código HTTP `201 Created`.

Parte 3: Especificaciones del Frontend (Web Components & CSS)

El frontend de la aplicación utilizará tecnologías nativas modernas. Deberá completar los siguientes esqueletos:

1. Elemento Tarjeta de Libro (`TarjetaLibro.js`)

Debe completar el componente `<tarjeta-libro>` que renderiza un libro en formato de tarjeta:

- **Shadow DOM:** Implemente Shadow DOM en modo abierto (`open`) para aislar el marcado y estilos del componente.
- **Templates y Slots:** Utilice una etiqueta `<template>` para definir la estructura HTML del componente. Utilice `<slot>` para colocar el título, autor, año y categoría del libro de manera dinámica.

- **Container Queries:** Agregue estilos CSS específicos dentro del Shadow DOM. Defina un contenedor y utilice una regla `@container` para que, cuando el contenedor de la tarjeta sea inferior a `350px` de ancho, la tarjeta se ordene en una sola columna vertical con fondo oscuro y fuentes pequeñas, y cuando sea mayor, se ordene en dos columnas horizontales.

2. Elemento Panel Biblioteca (`PanelBiblioteca.js`)

Debe completar el componente `<panel-biblioteca>` que controla el estado general de la vista:

- **Renderizado Base:** Construya la interfaz interna (un formulario para agregar libros y un contenedor `.catalogo-grid` para desplegar las tarjetas).
- **Consumo de Servicios (GET):** Implemente `cargarLibros()`. Debe realizar una petición asíncrona mediante `fetch` al backend REST, iterar por el JSON de respuesta e instanciar elementos `<tarjeta-libro>`. Transmite las propiedades a través de atributos `data-*` y añádalos a la retícula del catálogo.
- **Formulario y Envíos (POST):** Implemente `registrarLibro(event)`. Prevenga el comportamiento por defecto del formulario, capture los valores ingresados por el usuario, realice validaciones en el cliente (que los campos requeridos no estén en blanco) y envíe los datos al backend usando `fetch` con método `POST` serializado en JSON.
- **Actualización Dinámica:** Si el backend responde con un código de éxito `201 Created`, limpie los campos del formulario y ejecute nuevamente la carga de datos para reflejar el libro sin recargar la página entera.

3. Hoja de Estilos Global (`style.css`)

Estilice los componentes globales utilizando las siguientes pautas:

- **Grid Elástico:** Configure `.catalogo-grid` con la propiedad `display: grid`. Las columnas deben autoajustarse automáticamente utilizando `repeat(auto-fit, minmax(280px, 1fr))` para que el diseño sea responsivo sin usar Media Queries tradicionales.
- **Nesting CSS:** Utilice anidamiento nativo de CSS para estilizar el formulario, los inputs, las etiquetas y los estados del botón de guardar (ej. `hover` y `focus`), evitando escribir selectores repetitivos.

Parte 4: Flujo de Trabajo en Git e Integración

Para la entrega formal del laboratorio se evaluará el uso del control de versiones y la correcta integración del sistema.

Control de Versiones (Git)

- Inicialice un repositorio local si no lo ha hecho.
- Trabaje en una rama separada llamada `desarrollo/laboratorio2` para realizar sus cambios.
- Realice commits incrementales y estructurados siguiendo la especificación de **Conventional Commits**:
 - `feat(backend): ...` para cambios de controladores y dominios.
 - `feat(frontend): ...` para la creación y estilización de Web Components.
 - `fix(...): ...` para la resolución de errores detectados.
- Fusione sus cambios a la rama `main` de manera limpia al finalizar el laboratorio.

Parte 5: Pautas de Depuración

Durante el desarrollo es muy probable que se enfrente a problemas comunes de comunicación y renderizado. Utilice estas técnicas de diagnóstico:

1. **Conflicto de Puertos:** Si el backend no inicia e indica que el puerto 8080 está en uso, verifique las aplicaciones activas. Puede cambiar el puerto de Spring Boot en `src/main/resources/application.properties` agregando la línea `server.port=8081`, recordando actualizar también las peticiones `fetch` en el frontend.
2. **Ciclo de Vida de Web Components:** Recuerde que un Custom Element no puede auto-cerrarse en HTML (ej. `<tarjeta-libro />` es inválido). Siempre debe utilizar la etiqueta de cierre completa: `<tarjeta-libro></tarjeta-libro>`.
3. **Validación de Tipos de Datos:** Asegúrese de enviar el año de publicación como un número entero en el cuerpo del JSON (POST) y de configurar los encabezados de `fetch` con

'Content-Type': 'application/json'.

Rúbrica de Evaluación

La calificación definitiva de este laboratorio se obtendrá bajo los siguientes criterios de evaluación:

Criterio	Porcentaje	Descripción Detallada
Backend REST en Spring Boot	30%	Definición correcta del modelo, controlador REST con endpoints GET y POST, mapeo JSON correcto y habilitación de CORS.
Validaciones de Negocio	10%	Control de errores en el backend (retorno de HTTP 400 con mensaje) ante datos obligatorios vacíos o años inválidos.
Web Components en Frontend	25%	Implementación de <tarjeta-libro> con Shadow DOM y slots. Manejo del ciclo de vida y propiedades con data attributes.

Fetch API e Integración	15%	Peticiones asíncronas GET y POST desde <code><panel-biblioteca></code> , serialización JSON y actualización dinámica sin recargar la página.
CSS Moderno y Responsivo	10%	Uso correcto de CSS Nesting en la hoja de estilos, CSS Grid elástico y Container Queries en la tarjeta de libro.
Git, IA y Nomenclatura	10%	Commits semánticos y flujo en ramas. Estructura de paquetes según el estándar de nomenclatura del curso.
Total	100%	Nota final del Laboratorio 2